Panduan Teknis 01 - Memulai dengan Python dan Dasar Pemrograman

Dokumen ini ditujukan bagi pembaca yang belum pernah menggunakan Python atau baru mengenalnya. Tujuannya bukan menguasai seluruh bahasa Python, tetapi menyiapkan kemampuan minimum agar hands-on pada buku dapat dijalankan, dibaca, dan dimodifikasi dengan percaya diri.

> **Urutan yang akan dilakukan:** (1) memasang Python; (2) memeriksa instalasi; (3) membuat folder kerja; (4) menjalankan script pertama; (5) memahami sintaks dasar; (6) memasang library; (7) membaca error sederhana; dan (8) mencoba mini-program bertema robot.

Langkah 1 - Pasang Python

Python adalah program utama yang akan mengeksekusi file `.py`. Sebelum memasang library AI atau simulator, pastikan interpreter Python sudah tersedia.

Windows

1. Buka situs resmi **python.org** dan unduh Python 3 versi stabil.
2. Jalankan installer.
3. Aktifkan opsi **Add Python to PATH** jika tersedia.
4. Selesaikan instalasi, lalu buka PowerShell atau Command Prompt baru.

```
python --version
python -m pip --version
```

Jika perintah `python` tidak dikenali, coba `py --version`. Bila `py` bekerja, perintah berikut dapat memakai `py` sebagai pengganti `python`.

Ubuntu / Linux

Ubuntu biasanya sudah memiliki Python 3. Periksa terlebih dahulu:

```
python3 --version
pip3 --version
```

Jika belum tersedia:

```
sudo apt update
sudo apt install python3 python3-pip python3-venv
```

macOS

Python dapat dipasang melalui installer resmi atau Homebrew. Dengan Homebrew:

```
brew install python
python3 --version
```

> **Hasil yang diharapkan.** Terminal menampilkan versi Python dan pip tanpa error. Mulai dari titik ini, Python siap dipakai.

Langkah 2 - Buat Folder Kerja

Simpan latihan dalam satu folder agar file dan hasil eksperimen tidak bercampur dengan dokumen lain. Contoh:

```
embodied-ai-python/
```

Buka terminal pada folder tersebut. Untuk melihat lokasi aktif:

```
# Linux / macOS
pwd

# Windows
cd
```

Folder aktif penting karena perintah `python nama_file.py` mencari file relatif terhadap lokasi terminal saat itu.

Langkah 3 - Jalankan Program Python Pertama

Buat file bernama `hello_robot.py` menggunakan editor teks atau editor kode seperti Visual Studio Code. Isi:

```
print("Hello, Embodied AI!")
```

Simpan file, lalu jalankan:

```
python hello_robot.py
```

Di Linux/macOS, jika sistem menggunakan nama `python3`, jalankan:

```
python3 hello_robot.py
```

> **Jika berhasil,** terminal menampilkan `Hello, Embodied AI!`. Ini berarti kita sudah berhasil menulis, menyimpan, dan mengeksekusi sebuah script.

Langkah 4 - Kenali Variabel dan Tipe Data

Variabel menyimpan nilai yang akan digunakan oleh program. Dalam robotika, nilai tersebut dapat berupa posisi robot, nama objek, status gripper, jumlah step, atau hasil sensor.

```
robot_x = 0.25
robot_y = -0.10
object_name = "red cube"
gripper_closed = False
step_count = 12

print(robot_x)
print(object_name)
```

Tipe	Contoh	Contoh penggunaan
`int`	`12`	Jumlah step, indeks, counter
`float`	`0.25`	Posisi, reward, confidence
`str`	`"red cube"`	Nama objek atau instruction
`bool`	`True`	Status berhasil/gagal

Coba ubah nilai `robot_x`, simpan file, lalu jalankan lagi. Kebiasaan mengubah satu parameter dan melihat dampaknya akan sering dilakukan pada hands-on.

Langkah 5 - Simpan Beberapa Nilai dengan List dan Dictionary

`list` menyimpan beberapa nilai berurutan, sedangkan `dict` menyimpan pasangan key-value.

```
joint_positions = [0.0, -1.2, 0.8, 0.0]

robot_state = {{
    "x": 0.25,
    "y": -0.10,
    "gripper": "open",
}}

print(joint_positions[1])
print(robot_state["gripper"])
```

Pada project AI, list dan dictionary sering dipakai untuk menyimpan konfigurasi, observation, hasil evaluasi, atau parameter model.

Langkah 6 - Buat Keputusan dengan if

Robot atau program AI sering perlu menentukan tindakan berdasarkan kondisi tertentu.

```
distance = 0.08

if distance < 0.10:
    print("Object is close enough.")
else:
    print("Move closer.")
```

Perhatikan indentasi empat spasi setelah `if` dan `else`. Python memakai indentasi sebagai bagian dari struktur program.

Langkah 7 - Ulangi Proses dengan Loop

`for` cocok ketika jumlah pengulangan sudah diketahui. `while` cocok ketika proses diulang sampai kondisi tertentu terpenuhi.

```
for joint_id in range(4):
    print("Joint:", joint_id)

step = 0
while step < 3:
    print("Simulation step:", step)
    step += 1
```

> **Perhatikan `while`.** Selalu pastikan ada kondisi berhenti. Jika kondisinya tidak pernah menjadi salah, program dapat berjalan tanpa akhir.

Langkah 8 - Bungkus Logika dengan Fungsi

Fungsi membuat kode lebih mudah dipakai ulang dan dibaca.

```
def compute_distance(robot_x, target_x):
    distance = abs(target_x - robot_x)
    return distance

result = compute_distance(
    robot_x=0.20,
    target_x=0.75,
)

print("Distance:", result)
```

Pola ini akan sering muncul dalam fungsi preprocessing, perhitungan reward, pengambilan action, atau evaluasi hasil.

Langkah 9 - Gunakan Module dan Library

Python memiliki module bawaan dan dapat memakai library pihak ketiga.

```
import math

angle_degree = 90
angle_radian = math.radians(angle_degree)

print(angle_radian)
```

Library tambahan perlu dipasang terlebih dahulu. Contoh:

```
python -m pip install numpy matplotlib
```

Lalu gunakan di script:

```
import numpy as np

positions = np.array([0.1, 0.2, 0.3])
print(positions.mean())
```

> **Gunakan `python -m pip`.** Bentuk ini membantu memastikan pip memasang package ke interpreter Python yang sedang dipakai.

Langkah 10 - Belajar Membaca Error Dasar

Ketika program gagal, baca bagian paling bawah traceback terlebih dahulu. Tiga error dasar yang sering ditemui:

Error	Makna umum	Tindakan awal
`SyntaxError`	Struktur kode tidak valid.	Periksa tanda kurung, titik dua, atau penulisan sintaks.
`NameError`	Nama variabel/fungsi belum tersedia.	Periksa ejaan dan urutan eksekusi.
`ModuleNotFoundError`	Module belum terpasang atau environment salah.	Periksa pip dan environment aktif.

Contoh sengaja memicu `NameError`:

```
print(robot_speed)
```

Perbaiki dengan mendefinisikan variabel sebelum dipakai:

```
robot_speed = 0.10
print(robot_speed)
```

Langkah 11 - Coba Mini-Program Bertema Robot

Program berikut menggabungkan variabel, loop, kondisi, dan fungsi. Tujuannya adalah menggerakkan posisi satu dimensi mendekati target.

```
def move_toward_target(robot_x, target_x, step_size):
    distance = target_x - robot_x

    if abs(distance) < 0.01:
        return robot_x, True

    if distance > 0:
        robot_x += step_size
    else:
        robot_x -= step_size

    return robot_x, False


robot_x = 0.0
target_x = 0.5
step_size = 0.1

for step in range(10):
    robot_x, reached = move_toward_target(
        robot_x,
        target_x,
        step_size,
    )

    print(
        "Step:", step,
        "| robot_x:", round(robot_x, 2),
    )

    if reached:
        print("Target reached.")
        break
```

Alur program ini sudah menyerupai pola yang akan sering muncul dalam Embodied AI: membaca keadaan, menghitung keputusan, memperbarui keadaan, lalu memeriksa apakah tujuan tercapai.

Langkah 12 - Siap Melanjutkan ke Hands-on

Tidak perlu menghafal seluruh sintaks. Sebelum melanjutkan, yang paling penting adalah mampu melakukan empat hal: menjalankan script, mengubah parameter sederhana, mengikuti alur `if`/loop/fungsi, dan membaca bagian akhir traceback ketika terjadi error. Topik seperti NumPy, PyTorch, simulator, dan model AI akan diperkenalkan kembali saat benar-benar diperlukan.